

VisionInspectIA

AI-Powered Visual Inspection Platform for Bottle Defect Detection

Rapport de stage — Programme AI/ML Internship

Stagiaire :	Aymen Chiab
Organisme d'accueil :	TechSkillHub (programme à distance, www.techskillhub.tech)
Identifiant stagiaire :	TSH/A5E51B1E
Domaine :	AI / ML
Durée :	1 mois (4 semaines), à partir du 04 août 2026
Date de fin :	[À COMPLÉTER]
Encadrant / tuteur :	[À COMPLÉTER]
Établissement d'origine :	[À COMPLÉTER]
Année universitaire :	[À COMPLÉTER]

Remerciements

Je tiens à remercier TechSkillHub pour l'opportunité offerte de réaliser ce stage dans le domaine de l'intelligence artificielle appliquée à l'inspection visuelle industrielle, ainsi que pour la structuration claire du programme en quatre phases (préparation des données, modélisation, développement applicatif, déploiement) qui a guidé l'ensemble de la démarche.

Je remercie également [À COMPLÉTER] (encadrant(e) académique ou professionnel(le), le cas échéant) pour son suivi et ses conseils au cours de ce travail.

Enfin, je remercie [À COMPLÉTER] pour son soutien tout au long de cette période.

Résumé

VisionInspectIA est une plateforme web de détection automatique de défauts sur des bouteilles, développée dans le cadre d'un stage de quatre semaines au sein du programme AI/ML de TechSkillHub. Le projet répond à une problématique de contrôle qualité industriel : identifier automatiquement, à partir d'une photographie, si une bouteille est conforme ou présente un défaut, et le cas échéant en préciser la nature.

La solution retenue combine la vision par ordinateur et l'apprentissage profond par transfert (*transfer learning*). Quatre architectures ont été entraînées et comparées dans des conditions expérimentales identiques sur le dataset MVTec AD (classe *bottle*) : un CNN entraîné à partir de zéro, MobileNetV2, EfficientNetB0 et ResNet50. À l'issue de cette comparaison, MobileNetV2 a été retenu comme meilleur compromis entre performance (75,63 % d'accuracy, F1-score macro de 0,742 sur le jeu de test), taille du modèle (9,24 Mo) et temps d'inférence (environ 9 ms), MobileNetV2 étant nettement plus léger et plus rapide que ResNet50 pour une performance très proche.

Ce modèle a été intégré dans une application web complète : un backend FastAPI exposant une API REST (authentification JWT, upload d'image, prédiction, historique, statistiques, génération de rapports PDF), une base de données MySQL (utilisateurs et inspections), et un frontend React offrant un tableau de bord, une interface de prédiction, un historique filtrable, un export CSV, un mode sombre et un système de notifications.

L'application a ensuite été déployée réellement sur la plateforme Railway, avec un backend et un frontend hébergés comme deux services Docker distincts et une base MySQL managée. Le déploiement a été testé de bout en bout par des requêtes réelles contre les URLs publiques (authentification, prédiction sur les quatre classes, historique, tableau de bord, PDF, isolation des données entre utilisateurs, gestion des erreurs).

Les résultats obtenus montrent une bonne performance de détection pour les classes **good**, **broken_large** et **contamination**, mais une limitation identifiée sur la classe **broken_small**, régulièrement confondue avec **broken_large**, en raison de la rareté des images réelles disponibles pour cette classe. Une seconde limitation, propre à l'environnement de déploiement choisi, concerne le stockage des images uploadées, non persistant entre les redéploiements. Ces limites sont documentées et discutées, ainsi que les perspectives d'amélioration envisageables.

Abstract

VisionInspectIA is a web platform for automatic bottle defect detection, developed during a four-week internship within TechSkillHub’s AI/ML program. The project addresses an industrial quality-control problem: automatically determining, from a photograph, whether a bottle is defect-free or defective, and if so, identifying the type of defect.

The retained solution combines computer vision and transfer learning. Four architectures were trained and compared under identical experimental conditions on the MVTec AD dataset (*bottle* category): a CNN trained from scratch, MobileNetV2, EfficientNetB0, and ResNet50. Following this comparison, MobileNetV2 was selected as the best trade-off between performance (75.63% test accuracy, 0.742 macro F1-score), model size (9.24 MB), and inference time (about 9 ms), being substantially lighter and faster than ResNet50 for a very close performance.

This model was integrated into a full web application: a FastAPI backend exposing a REST API (JWT authentication, image upload, prediction, history, statistics, PDF report generation), a MySQL database (users and inspections), and a React frontend providing a dashboard, a prediction interface, a filterable history, CSV export, dark mode, and a notification system.

The application was then genuinely deployed on the Railway platform, with the backend and frontend hosted as two separate Docker services and a managed MySQL database. The deployment was tested end-to-end with real requests against the public URLs (authentication, prediction on all four classes, history, dashboard, PDF, cross-user data isolation, error handling).

Results show good detection performance for the **good**, **broken_large**, and **contamination** classes, but a limitation was identified for the **broken_small** class, frequently confused with **broken_large**, due to the scarcity of real images available for this class. A second, deployment-specific limitation concerns uploaded image storage, which does not persist across redeployments. These limitations are documented and discussed, along with possible directions for future improvement.

Table des matières

Remerciements	i
Résumé	ii
Abstract	iii
1 Contexte et problématique	1
1.1 Contexte du projet	1
1.2 Contexte de l’inspection visuelle	1
1.3 Problématique	1
1.4 Besoin métier	1
1.5 Objectifs	2
1.6 Cahier des charges	2
1.7 Fonctionnalités attendues	2
1.8 Contraintes	3
1.9 Méthodologie	3
1.10 Organisation du projet	3
2 Dataset et préparation des données	4
2.1 Présentation du dataset	4
2.2 MVTec AD	4
2.3 Classe bottle	4
2.4 Classes de défauts	4
2.5 Analyse exploratoire	5
2.6 Nettoyage	5
2.7 Vérification des doublons	5
2.8 Répartition des données	6
2.9 Train / Validation / Test	6
2.10 Redimensionnement 224×224	6
2.11 Preprocessing	6
2.12 Data augmentation	6
2.13 Dataset augmenté	7
2.14 Problème de data leakage	8
2.15 Dataset final	9

3	Modélisation IA	10
3.1	Problème de classification	10
3.2	CNN de base	10
3.3	Transfer learning	10
3.4	MobileNetV2	10
3.5	EfficientNetB0	10
3.6	ResNet50	11
3.7	Méthodologie d'entraînement	11
3.8	Fonction de perte	11
3.9	Optimiseur	11
3.10	Callbacks	11
3.11	Évaluation	11
3.12	Benchmark	12
3.13	Analyse des résultats	14
3.14	Sélection de MobileNetV2	15
3.15	Modèle final	15
3.16	Limites	15
4	Architecture de la solution	16
4.1	Architecture globale	16
4.2	Frontend	16
4.3	Backend	17
4.4	IA	17
4.5	Base de données	17
4.6	Authentification	17
4.7	Communication frontend/backend	17
4.8	Gestion des fichiers	17
4.9	Sécurité	17
5	Base de données	18
5.1	Choix de MySQL	18
5.2	Modèle de données	18
5.3	Contraintes	19
5.4	Isolation des utilisateurs	19
5.5	Gestion des inspections	19
5.6	Déploiement de MySQL sur Railway	20
6	Backend FastAPI	21
6.1	Présentation de FastAPI	21
6.2	Architecture backend	21
6.3	Configuration	21
6.4	Authentification JWT	21
6.5	Gestion des utilisateurs	22

6.6	Upload	22
6.7	Pipeline de prédiction	22
6.8	Historique	22
6.9	Dashboard	22
6.10	PDF	22
6.11	Sécurité	22
6.12	API REST	23
7	Frontend React	24
7.1	Choix de React	24
7.2	Architecture frontend	24
7.3	Routing	24
7.4	AuthContext	24
7.5	Axios	24
7.6	Login / Register	25
7.7	Dashboard	25
7.8	Prediction	25
7.9	History	25
7.10	Profile	25
7.11	PDF et CSV	25
7.12	Notifications	25
7.13	Dark mode	25
7.14	Responsive design	25
7.15	UX/UI — limite importante	25
7.16	Fonctionnalités de la plateforme	26
8	Pipeline de prédiction IA	27
9	Tests et validation	30
9.1	Tests backend	30
9.2	Tests frontend	30
9.3	Tests API	30
9.4	Tests IA	30
9.5	Tests sécurité	30
9.6	Tests base de données	30
9.7	Tests PDF	30
9.8	Tests CSV	31
9.9	Tests end-to-end	31
9.10	Tests en production	31
10	Déploiement	32
10.1	Pourquoi Railway	32
10.2	Architecture de production	32
10.3	Docker	33

10.4 Backend	33
10.5 Frontend	33
10.6 MySQL	33
10.7 Variables d'environnement	33
10.8 MobileNetV2 en production	33
10.9 URLs	34
10.10 Tests en production	34
10.11 Performances	34
10.12 Problèmes rencontrés	34
10.13 Solutions appliquées	35
11 Résultats, discussion, limitations et perspectives	36
11.1 Résultats IA	36
11.2 Benchmark	36
11.3 Analyse par classe	36
11.4 Résultats applicatifs et production	37
11.5 Discussion	37
11.6 Limitations	38
11.7 Perspectives	38
12 Conclusion générale	39
Bibliographie	40
A Architecture globale	41
B Endpoints de l'API	42
C Base de données	43
D Résultats IA	44
E Captures de la plateforme	45
F Déploiement	46

Table des figures

2.1	Distribution du dataset brut (292 images, MVTec AD catégorie bottle).	5
2.2	Techniques d’augmentation réellement implémentées dans le pipeline.	7
2.3	Distribution du dataset augmenté (train/test).	8
3.1	Courbes d’entraînement de MobileNetV2 (source : <code>ai/results/mobilenet_v2/</code>).	13
3.2	Matrice de confusion de MobileNetV2 sur le jeu de test.	13
3.3	Precision, recall et F1-score par classe pour MobileNetV2.	14
3.4	Comparaison des quatre architectures : F1-score macro, temps d’inférence, taille du modèle.	14
4.1	Architecture globale de la plateforme VisionInspectIA.	16
5.1	Schéma de la base de données MySQL (relation <code>users</code> 1–N <code>inspections</code>).	18
8.1	Pipeline interne du modèle MobileNetV2 (<code>best_model.keras</code>).	27
8.2	Pipeline de prédiction : flux applicatif complet depuis l’upload jusqu’à la réponse.	28
10.1	Architecture de déploiement en production sur Railway.	32
A.1	Rappel : architecture globale de la plateforme VisionInspectIA.	41

Liste des tableaux

2.1	Classes du dataset (données brutes)	4
2.2	Répartition Train / Validation / Test (dataset augmenté)	9
3.1	Hyperparamètres d'entraînement	11
3.2	Benchmark comparatif des modèles (jeu de test, 160 images, 40 par classe)	12
3.3	Classification report détaillé (MobileNetV2, modèle retenu)	12
5.1	Table <code>users</code> (source : <code>backend/app/models/user.py</code>)	19
5.2	Table <code>inspections</code> (source : <code>backend/app/models/inspection.py</code>)	19
6.1	Endpoints de l'API REST (source : <code>backend/app/api/v1/router.py</code>)	23
7.1	Fonctionnalités implementées et leur statut de vérification	26
9.1	Tests en production (requêtes HTTP réelles contre les URLs publiques de déploiement)	31
11.1	Limitations identifiées	38
D.1	Correspondance exigences du cahier des charges / réalisation	44

CHAPITRE 1

Contexte et problématique

1.1 Contexte du projet

Ce projet a été réalisé dans le cadre d'un stage d'un mois au sein du programme AI/ML de TechSkillHub, organisme proposant des stages structurés à distance dans les domaines de l'intelligence artificielle et du développement full-stack. Le sujet assigné, intitulé « *VisionInspect AI – Intelligent Product Defect Detection System* », consiste à développer une plateforme de contrôle qualité industriel capable de détecter automatiquement des défauts sur des produits à partir d'images, en s'appuyant sur le Deep Learning et la vision par ordinateur.

1.2 Contexte de l'inspection visuelle

Dans l'industrie manufacturière, le contrôle qualité visuel reste, dans de nombreux contextes, réalisé manuellement par des opérateurs humains. Cette approche présente des limites connues : fatigue visuelle, subjectivité, variabilité entre opérateurs, et coût difficilement compressible à grande échelle. L'automatisation de cette tâche par des méthodes de vision par ordinateur constitue un axe de recherche appliquée actif, particulièrement depuis la démocratisation des réseaux de neurones convolutifs profonds et des techniques de *transfer learning*, qui permettent d'obtenir des performances satisfaisantes même avec un volume de données d'entraînement limité.

1.3 Problématique

La problématique traitée dans ce projet est la suivante : *comment détecter automatiquement, à partir d'une simple photographie, si une bouteille présente un défaut, et le cas échéant identifier le type de défaut, dans un contexte où le nombre d'images réelles disponibles pour l'entraînement est restreint ?*

1.4 Besoin métier

Le besoin métier sous-jacent, tel que formulé dans le cahier des charges du programme [9], est de fournir à une entreprise manufacturière un outil permettant :

— d'uploader une image de produit ;

- d’obtenir automatiquement une classification (conforme / type de défaut) avec un score de confiance ;
- de conserver un historique des inspections réalisées ;
- de disposer d’un tableau de bord analytique ;
- de générer des rapports d’inspection téléchargeables.

1.5 Objectifs

Les objectifs du stage, tels que définis par le document de cadrage, étaient organisés en quatre phases sur quatre semaines :

1. Préparer un dataset exploitable pour l’entraînement d’un modèle de classification d’images (nettoyage, augmentation, analyse exploratoire).
2. Entraîner et comparer plusieurs architectures de Deep Learning, puis sélectionner la plus adaptée.
3. Développer une application web complète (frontend + backend + base de données) intégrant ce modèle.
4. Déployer la plateforme sur une infrastructure cloud réelle et produire la documentation associée.

1.6 Cahier des charges

Le cahier des charges, extrait du document de cadrage TechSkillHub, précise les éléments suivants :

- **Dataset** : au choix parmi Kaggle, Roboflow, MVTec AD, ou NEU Surface Defect.
- **Modèles à comparer** : CNN personnalisé, MobileNetV2, EfficientNetB0, ResNet50.
- **Métriques d’évaluation** : Accuracy, Precision, Recall, F1-score, courbe ROC, matrice de confusion.
- **Stack applicative** : Frontend React ou Angular ; Backend FastAPI ou Flask ; Base de données MySQL.
- **Fonctionnalités clés** : authentification et tableau de bord, upload d’image avec score de confiance, historique des prédictions, export PDF.
- **Déploiement** : sur Render, Railway, ou Hugging Face Spaces.
- **Livrables finaux** : dépôt GitHub, modèle entraîné, API et application web, schéma de base de données et captures d’écran, rapport de projet (PDF) et présentation, diagramme d’architecture, vidéo de démonstration de 5 à 10 minutes.

1.7 Fonctionnalités attendues

Le document de cadrage liste explicitement : détection de produits défectueux, classification du type de défaut, scores de confiance, historique des inspections, rapports téléchargeables,

tableau de bord analytique.

1.8 Contraintes

Les principales contraintes identifiées au cours du projet sont :

- un volume de données réelles restreint (dataset MVTec AD, catégorie *bottle*, quelques dizaines d’images réelles par classe — détaillé au 2) ;
- une durée de réalisation limitée à quatre semaines, couvrant à la fois la partie recherche (dataset, modèles) et la partie ingénierie logicielle (application web complète, déploiement) ;
- l’absence d’environnement de test disposant d’un navigateur graphique, limitant la vérification visuelle de l’interface à une vérification manuelle a posteriori (voir 7 et 9).

1.9 Méthodologie

La méthodologie suivie correspond à une démarche d’ingénierie itérative : analyse du problème, expérimentation contrôlée (protocole identique pour les quatre architectures comparées), évaluation quantitative sur un jeu de test dédié, sélection argumentée du modèle, implémentation, tests fonctionnels puis tests en conditions de production réelles, et documentation explicite des limitations rencontrées à chaque étape plutôt que leur dissimulation.

1.10 Organisation du projet

Le projet est organisé en quatre grands modules correspondant aux quatre phases du programme :

```
1 ai/          -> Recherche : preparation du dataset, entraînement,  
    benchmark  
2 backend/     -> API REST FastAPI  
3 frontend/    -> Application React  
4 docs/        -> Documentation technique et rapport de stage
```

Incohérence à vérifier : Le document de cadrage TechSkillHub propose, à titre indicatif, une arborescence de dépôt légèrement différente (`dataset/`, `models/`, `frontend/`, `backend/`, `api/`, `reports/`, `documentation/`, `README.md`). L’arborescence effectivement retenue pour ce projet regroupe la partie recherche/IA dans un seul dossier `ai/` (incluant `dataset`, modèles et résultats plutôt que des dossiers séparés `dataset/`/`models/`), n’isole pas de dossier `api/` distinct de `backend/` (les routes API étant un sous-module de `backend/app/api/`), et nomme `docs/` le dossier de documentation plutôt que `documentation/`. Cette réorganisation a été jugée plus cohérente avec la séparation en couches du backend (6) mais s’écarte de la structure indicative du cahier des charges ; elle est signalée ici plutôt que présentée comme strictement conforme.

CHAPITRE 2

Dataset et préparation des données

2.1 Présentation du dataset

Le dataset retenu parmi les options proposées par le cahier des charges (Kaggle, Roboflow, MVTec AD, NEU Surface Defect) est **MVTec AD** [5], un jeu de données de référence pour la détection d'anomalies industrielles.

2.2 MVTec AD

MVTec AD (*MVTec Anomaly Detection Dataset*) est un dataset public conçu spécifiquement pour l'évaluation de méthodes de détection d'anomalies dans un contexte industriel. Il regroupe plusieurs catégories d'objets et de textures, chacune avec un jeu d'images « normales » (sans défaut) destinées à l'entraînement et un jeu d'images de test réparties entre une classe normale et plusieurs classes de défauts.

2.3 Classe bottle

Ce projet utilise exclusivement la catégorie **bottle** du dataset MVTec AD.

2.4 Classes de défauts

Le jeu de données brut (source : `ai/results/reports/dataset_report.json`) est structuré comme suit :

TABLE 2.1 – Classes du dataset (données brutes)

Répertoire source	Nombre d'images
<code>train/good</code>	209
<code>test/good</code>	20
<code>test/broken_large</code>	20
<code>test/broken_small</code>	22
<code>test/contamination</code>	21
Total	292

Cette structure est typique des datasets MVTec AD : le répertoire `train/` ne contient que des images conformes (`good`), tandis que les images défectueuses n'existent que dans `test/`. Les quatre classes retenues pour la classification finale sont : `good`, `broken_large`, `broken_small`, `contamination`.

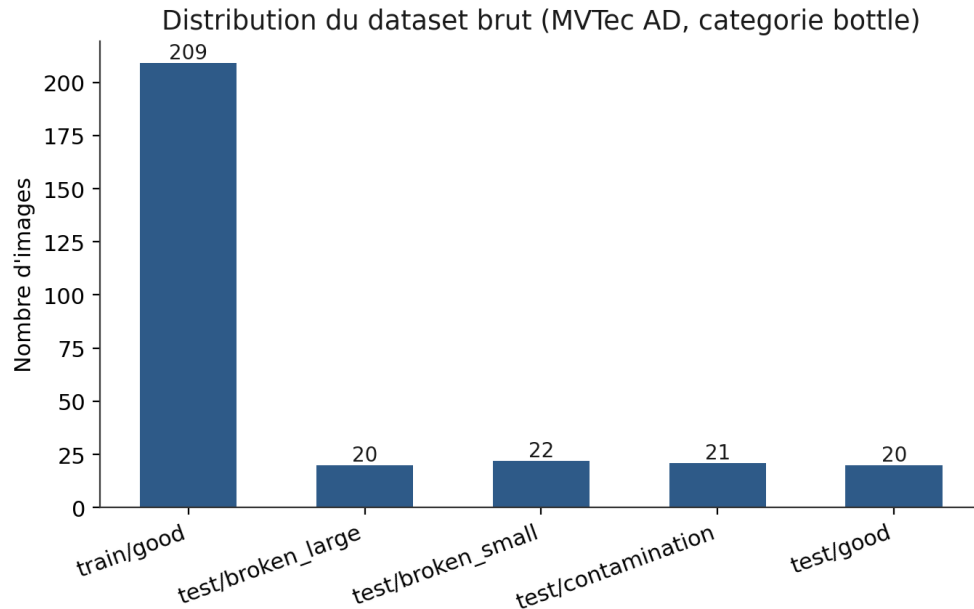


Figure 5a - Repartition des 292 images brutes par repertoire source (source : `ai/results/reports/dataset_report.json`).

FIGURE 2.1 – Distribution du dataset brut (292 images, MVTec AD catégorie bottle).

2.5 Analyse exploratoire

L'analyse exploratoire, réalisée automatiquement, a porté sur :

- le format des fichiers : 292 images, toutes au format `.png` ;
- les dimensions : toutes les images sont de taille homogène, 900×900 pixels ;
- le mode colorimétrique : RGB (3 canaux) pour l'ensemble des 292 images ;
- l'intégrité des fichiers : 0 image corrompue détectée sur les 292 analysées ;
- les doublons : 0 doublon détecté par le script d'analyse (`duplicate_images`: 0).

2.6 Nettoyage

Le script d'analyse n'a signalé aucune image corrompue ni aucun doublon exact sur le jeu brut. Le nettoyage à ce stade s'est donc limité à une vérification de conformité (format, intégrité), sans suppression d'images.

2.7 Vérification des doublons

La vérification automatique des doublons exacts sur les 292 images sources n'a révélé aucune occurrence. Cette vérification porte sur les images brutes ; la problématique distincte de similarité

entre images augmentées (traitée en 2.14) a nécessité une analyse séparée.

2.8 Répartition des données

Compte tenu du faible nombre d'images réelles disponibles par classe de défaut (20 à 22 images), en particulier pour l'entraînement où seules des images `good` existent nativement dans MVTec AD, une étape d'augmentation a été nécessaire pour construire un jeu d'entraînement équilibré entre les quatre classes.

2.9 Train / Validation / Test

Le jeu de données a été réparti en trois ensembles disjoints (train, validation, test) avant l'étape d'augmentation, en veillant à ce qu'aucune image source ne soit partagée entre plusieurs ensembles (voir 2.14).

2.10 Redimensionnement 224×224

Toutes les images sont redimensionnées en 224×224 pixels avant d'être fournies au modèle, taille d'entrée standard pour les architectures MobileNetV2, EfficientNetB0 et ResNet50 utilisées dans ce projet (source : `backend/app/ml/preprocessing.py`, `ai/config/config.py`).

2.11 Preprocessing

Le pipeline de prétraitement, identique à l'entraînement et à l'inférence en production, est le suivant :

1. décodage de l'image en RGB (3 canaux) ;
2. redimensionnement bilinéaire en 224×224 pixels ;
3. conversion en `float32`, valeurs dans l'intervalle $[0, 255]$.

La normalisation propre à MobileNetV2 ($(x/127,5-1,0)$) **n'est pas appliquée à cette étape**. Elle est directement intégrée au graphe du modèle sauvegardé, sous la forme d'une couche personnalisée `MobileNetPreprocess` (source : `ai/models/preprocessing_layers.py`, dupliquée dans `backend/app/ml/preprocessing_layers.py` pour l'autonomie du backend en production — voir 10). Ce choix de conception a été vérifié explicitement afin d'éviter qu'une double normalisation ne soit appliquée à l'image (ce qui aurait faussé les prédictions).

2.12 Data augmentation

Les techniques d'augmentation réellement implémentées dans le pipeline (source : `ai/scripts/data_augmentation.py`, méthode `build_pipeline`) reposent sur les couches Keras suivantes :

- `RandomFlip` (retournement horizontal/vertical) ;
- `RandomRotation` (rotation aléatoire) ;

- **RandomZoom** (zoom aléatoire) ;
- **RandomBrightness** (variation aléatoire de luminosité).

Le cahier des charges du programme mentionnait également des techniques de contraste et de recadrage (*Contrast*, *Crop*) parmi les augmentations possibles ; ces deux techniques spécifiques ne sont pas retrouvées dans le pipeline implémenté et ne sont donc pas revendiquées ici.

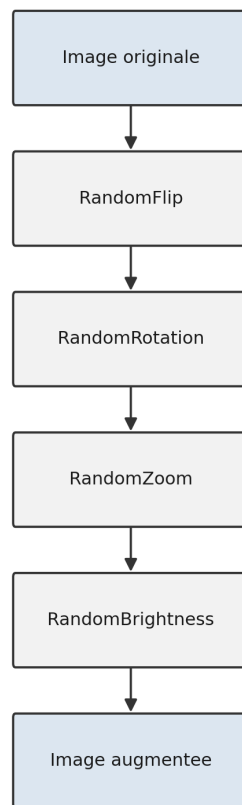


Figure 6 - Techniques d'augmentation réellement implémentées
(source : ai/scripts/data_augmentation.py, methode build_pipeline).
Aucune autre transformation (contraste, recadrage) n'est appliquée.

FIGURE 2.2 – Techniques d’augmentation réellement implémentées dans le pipeline.

2.13 Dataset augmenté

L’augmentation a été appliquée de manière différenciée selon l’ensemble concerné :

- **Train** : complété par duplication/augmentation jusqu’à une cible uniforme de 200 images par classe.
- **Test** : complété de la même manière jusqu’à une cible de 40 images par classe.
- **Validation** : traité différemment par choix de conception explicite — voir 2.14.

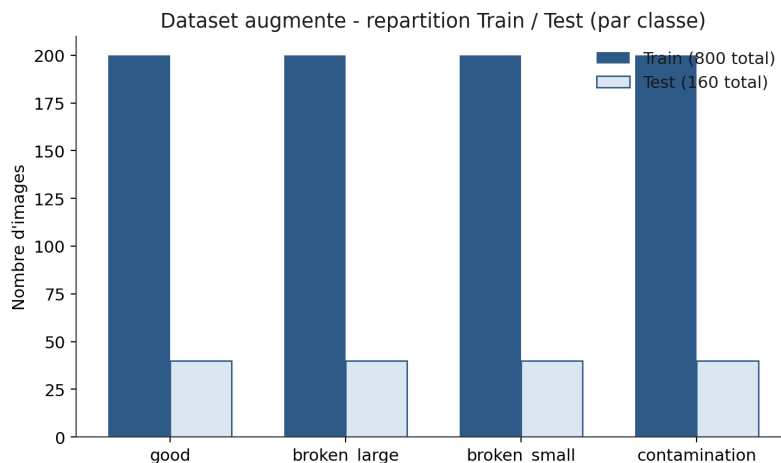


Figure 5b - Dataset augmente equilibre : 200 images/classe en train, 40 images/classe en test (source : ai/results/reports/augmentation_report.json). La validation n'est pas representee ici car non uniforme entre classes (voir Chapitre 2.15 du rapport).

FIGURE 2.3 – Distribution du dataset augmenté (train/test).

2.14 Problème de data leakage

Problème initial. Une première approche consistait à compléter (« padder ») l'ensemble de validation par duplication, à l'instar de l'ensemble de test, afin d'atteindre un nombre cible uniforme d'images par classe.

Détection du problème. Cette approche a été identifiée comme risquée : un signal de validation basé sur des images dupliquées (quasi identiques à des images du jeu d'entraînement après augmentation) est bruité et peut fausser les décisions prises pendant l'entraînement sur la base de ce signal, en particulier l'*early stopping* et la sélection du meilleur point de sauvegarde (*checkpoint*) du modèle. Cette analyse est documentée explicitement dans le code source (ai/scripts/data_augmentation.py, méthode `balance_split`) :

« pad=False : contrairement au test, la validation n'est jamais complétée par des copies dupliquées. Un signal de validation basé sur des doublons est bruité et fausse les décisions d'early stopping / sélection du meilleur checkpoint pendant l'entraînement. On garde donc uniquement les images réelles uniques disponibles, quitte à avoir moins que la cible. »

Correction appliquée. L'ensemble de validation est donc construit exclusivement à partir d'images réelles uniques, sans complétion par duplication, quitte à obtenir un nombre d'images inférieur à la cible nominale de 40 par classe pour les classes disposant de peu d'images sources.

Protocole final. Le jeu de test, en revanche, est complété par duplication pour atteindre 40 images par classe de façon homogène, ce choix étant jugé acceptable dans la mesure où le test n'intervient pas dans les décisions prises pendant l'entraînement.

Preuve indirecte de la correction. Le dépôt conserve plusieurs *runs* expérimentaux de MobileNetV2 permettant de tracer cette évolution :

- ai/results/mobilenet_v2_baseline_padded_val/ (09/08/2026, 02 :16) : validation composée de 160 images (40 par classe — cohérent avec une validation *paddée*);
- ai/results/mobilenet_v2_exp0_fixed_val/ (09/08/2026, 17 :06) : validation réduite à 64 images (*fixed_val*, cohérent avec la correction décrite ci-dessus);

- `ai/results/mobilenet_v2/` (run final retenu, 09/08/2026, 17 :18) : validation également composée de 64 images, confirmant que le modèle retenu a bien été entraîné selon le protocole corrigé.

2.15 Dataset final

TABLE 2.2 – Répartition Train / Validation / Test (dataset augmenté)

Ensemble	Images totales	Par classe (cible)
Train	800	200
Test	160	40
Validation	64 (modèle retenu)	non uniforme, voir ci-dessous

Incohérence à vérifier : Le fichier `ai/results/reports/augmentation_report.json` (généré le 09/08/2026 à 17 :13) rapporte une composition de validation de **55 images** (40 pour `good`, 5 pour chacune des trois classes de défaut), alors que le fichier `training_report.json` du modèle MobileNetV2 finalement retenu (généré à 17 :18, soit peu après) indique **64 images** de validation. L'écart de 9 images entre ces deux fichiers, générés à quelques minutes d'intervalle, n'a pas pu être expliqué à partir des fichiers disponibles dans le dépôt ; il est possible qu'une régénération partielle du split de validation soit intervenue entre les deux exécutions. Cette incohérence est signalée ici plutôt que résolue arbitrairement.

Toutes les architectures comparées au 3 n'ont pas nécessairement été entraînées avec la même taille de validation : les *runs* de ResNet50, EfficientNetB0, CNN et CNN amélioré rapportent chacun 160 images de validation, contre 64 pour le run MobileNetV2 finalement retenu. Cette différence de protocole entre modèles comparés constitue une limite du benchmark, discutée au 11.

CHAPITRE 3

Modélisation IA

3.1 Problème de classification

Il s'agit d'un problème de classification d'images multi-classes (4 classes), à sortie unique (chaque image appartient à exactement une classe).

3.2 CNN de base

Un réseau de neurones convolutif (CNN) construit et entraîné intégralement à partir de zéro (sans poids pré-entraînés) a été inclus dans la comparaison, afin de disposer d'une référence (*baseline*) ne bénéficiant d'aucun transfert de connaissances.

3.3 Transfer learning

Les trois autres architectures comparées (MobileNetV2, EfficientNetB0, ResNet50) exploitent le *transfer learning* : un backbone pré-entraîné sur ImageNet est réutilisé, gelé (`trainable=False`), seule une tête de classification ajoutée en sortie étant entraînée sur le dataset *bottle* (protocole dit de *feature extraction*).

3.4 MobileNetV2

Architecture légère, fondée sur des convolutions séparables en profondeur (*depthwise separable convolutions*), initialement conçue pour des déploiements mobiles/embarqués. Retenue au final pour ce projet (justification détaillée en 3.14) [7].

3.5 EfficientNetB0

Architecture fondée sur un principe de *compound scaling* (mise à l'échelle simultanée de la profondeur, de la largeur et de la résolution du réseau), incluse dans la comparaison [8].

3.6 ResNet50

Architecture à connexions résiduelles, plus profonde et plus lourde que les deux précédentes (23,6 millions de paramètres contre 2,26 pour MobileNetV2), également incluse dans la comparaison [6].

3.7 Méthodologie d'entraînement

TABLE 3.1 – Hyperparamètres d'entraînement

Paramètre	MobileNetV2 / ResNet50 / EffNetB0	CNN / CNN amélioré
Époques (max.)	30	30
Taille de batch	32	32
Taux d'apprentissage	0,001	0,001
Optimiseur	Adam	AdamW
Dropout	0,3	0,3
Weight decay	0,0001	0,0001
Taille d'image	224×224	224×224
Nombre de classes	4	4
Graine aléatoire (seed)	42	42

3.8 Fonction de perte

`SparseCategoricalCrossentropy`, utilisée de façon identique pour les quatre architectures (source : `ai/models/mobilenet_model.py`, `resnet_model.py`, `efficientnet_model.py`, `cnn_model.py`).

3.9 Optimiseur

Adam pour MobileNetV2, ResNet50 et EfficientNetB0 ; AdamW pour le CNN personnalisé et sa variante améliorée.

3.10 Callbacks

- **EarlyStopping** : patience de 8 époques, surveillance de `val_loss` ;
- **ModelCheckpoint** : sauvegarde du meilleur modèle selon `val_loss` (`save_best_only=True`).

3.11 Évaluation

Les métriques calculées pour chaque modèle sur le jeu de test sont : accuracy, precision macro, recall macro, F1-score macro, ainsi qu'une matrice de confusion.

Donnée non vérifiable : Le cahier des charges demande également une courbe ROC par modèle ; aucun fichier correspondant à une courbe ROC n’a été retrouvé dans `ai/results/`. Cette visualisation n’a donc pas pu être produite dans le cadre de ce rapport.

3.12 Benchmark

TABLE 3.2 – Benchmark comparatif des modèles (jeu de test, 160 images, 40 par classe)

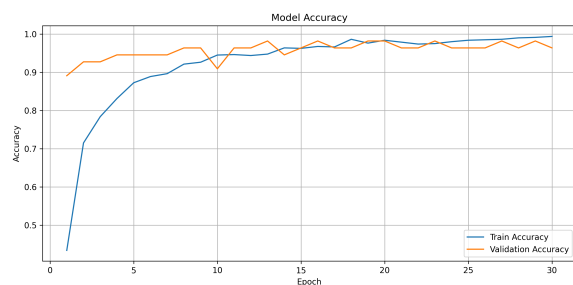
Modèle	Acc.	Prec.	Rec.	F1	Taille	T. entraî.	Inférence
MobileNetV2 (retenu)	75,63 %	0,839	0,756	0,742	9,24 Mo	3 min 57 s	9,16 ms
ResNet50	78,13 %	0,818	0,781	0,772	90,71 Mo	10 min 17 s	23,70 ms
EfficientNetB0	71,25 %	0,826	0,713	0,688	16,32 Mo	5 min 56 s	11,45 ms
CNN (from scratch)	25,00 %	0,063	0,250	0,100	21,68 Mo	≈8 min 40 s	7,4–7,8 ms

Incohérence à vérifier : Le fichier `ai/results/benchmark_results.json` comporte, pour chaque modèle, un champ `accuracy` de premier niveau dont la valeur ne correspond pas à celle du fichier `classification_report.json` du même modèle (exemple : 99,37 % pour MobileNetV2 dans `benchmark_results.json`, contre 75,63 % dans `classification_report.json`, ce dernier étant cohérent avec le F1-score macro rapporté de 0,742). Cette incohérence a été résolue en retenant les valeurs de `classification_report.json` comme référence, celles-ci étant seules cohérentes en interne avec les autres métriques macro rapportées.

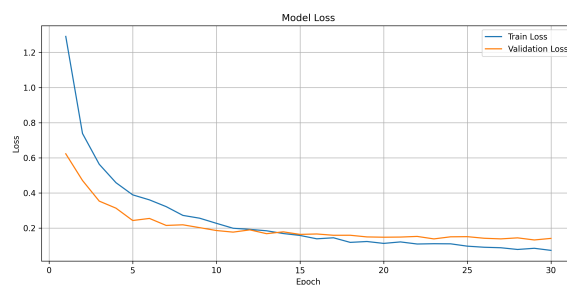
Résultat d’un benchmark antérieur, sur dataset non augmenté. Un premier benchmark (`ai/results/benchmark_report.md`, daté du 08/08/2026), réalisé sur le dataset brut de 292 images (avant la construction du dataset augmenté équilibré), avait donné des résultats très dégradés et strictement identiques pour les quatre architectures (F1-score macro de 0,2179 pour chacune), le rapport concluant à un sous-apprentissage généralisé lié à l’insuffisance de données et au protocole de *feature extraction* à backbone gelé sur un jeu de données aussi réduit. Ce benchmark constitue une itération intermédiaire du projet, corrigée par la suite via la construction du dataset augmenté équilibré, et n’est pas représentatif de la performance du modèle finalement intégré à l’application.

TABLE 3.3 – Classification report détaillé (MobileNetV2, modèle retenu)

Classe	Precision	Recall	F1-score	Support
<code>broken_large</code>	0,741	1,000	0,851	40
<code>broken_small</code>	1,000	0,500	0,667	40
<code>contamination</code>	1,000	0,525	0,689	40
<code>good</code>	0,615	1,000	0,762	40



(a) Accuracy pendant l'entraînement



(b) Loss pendant l'entraînement

FIGURE 3.1 – Courbes d'entraînement de MobileNetV2 (source : ai/results/mobilenet_v2/).

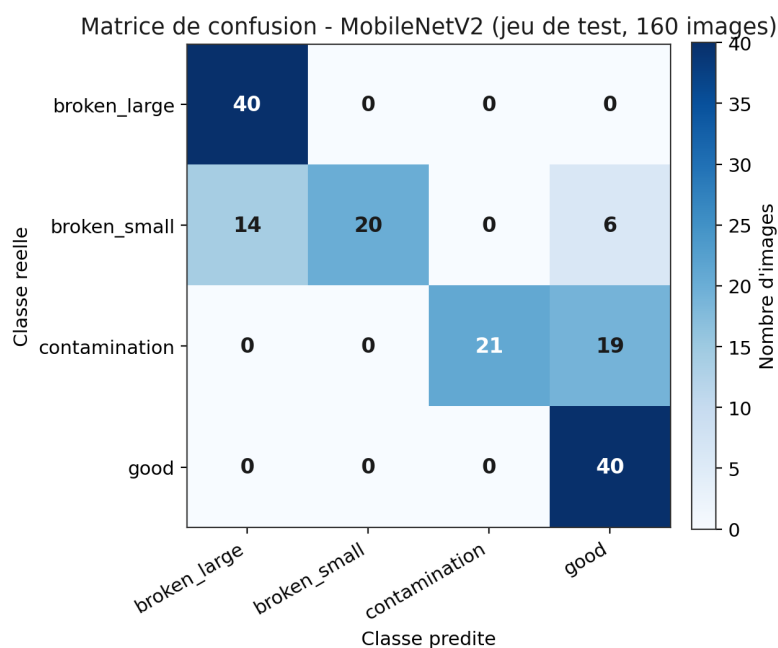


Figure 9 - Matrice de confusion de MobileNetV2 sur le jeu de test
 (source : ai/results/mobilenet_v2/confusion_matrix.npy, valeurs non modifiées).

FIGURE 3.2 – Matrice de confusion de MobileNetV2 sur le jeu de test.

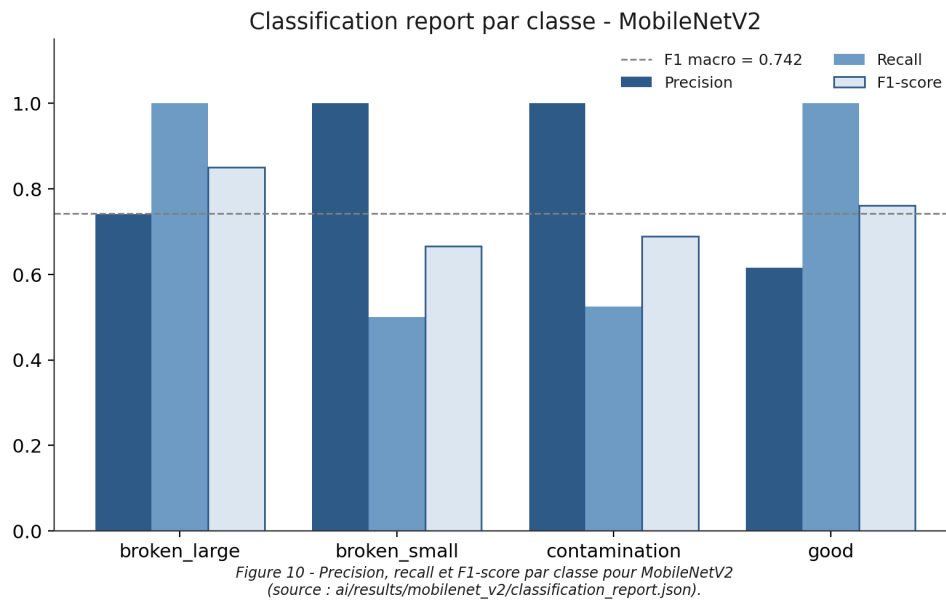


FIGURE 3.3 – Precision, recall et F1-score par classe pour MobileNetV2.

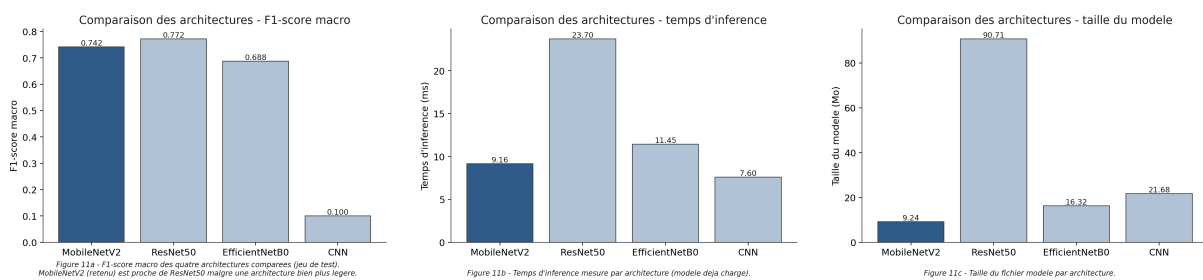


FIGURE 3.4 – Comparaison des quatre architectures : F1-score macro, temps d'inférence, taille du modèle.

3.13 Analyse des résultats

Les trois modèles pré-entraînés sur ImageNet (MobileNetV2, EfficientNetB0, ResNet50) surpassent nettement le CNN entraîné à partir de zéro, dont l'accuracy (25 %) reste proche du niveau du hasard pour un problème à quatre classes équilibrées, ce qui met en évidence l'apport du transfer learning dans un contexte de données limitées.

Entre les trois modèles pré-entraînés, ResNet50 obtient l'accuracy la plus élevée (78,13 % contre 75,63 % pour MobileNetV2), mais cet écart de 2,5 points doit être interprété avec prudence : le jeu de test ne comporte que quelques images sources réellement uniques par classe (le reste étant obtenu par duplication/augmentation), ce qui limite la robustesse statistique d'un tel écart.

Le 3.3 met en évidence un comportement différencié selon les classes pour MobileNetV2 : un rappel parfait (1,0) sur **broken_large** et **good**, mais un rappel faible (0,5 et 0,525) sur **broken_small** et **contamination**, ces deux classes étant plus difficiles à distinguer visuellement dans ce dataset.

3.14 Sélection de MobileNetV2

Dans les conditions expérimentales de ce projet, **MobileNetV2 a été retenu comme meilleur compromis entre performance, taille du modèle et temps d'inférence**, plutôt que comme le modèle objectivement le plus performant :

- **Performance** : F1-score macro de 0,742, proche de celui de ResNet50 (0,772) et supérieur à celui d'EfficientNetB0 (0,688).
- **Taille du modèle** : 9,24 Mo, soit environ 10 fois plus léger que ResNet50 (90,71 Mo).
- **Rapidité d'inférence** : environ 9 ms, soit 2,6 fois plus rapide que ResNet50 (23,70 ms).
- **Facilité de déploiement** : ces caractéristiques rendent le modèle mieux adapté à une intégration dans une application web sans infrastructure GPU dédiée (voir 10).

3.15 Modèle final

Le modèle final retenu est sauvegardé au format natif Keras 3 (`.keras`) : `ai/saved_models/mobilenet_v2/best_model.keras`. Une copie de ce même fichier (identité vérifiée par empreinte SHA-256, voir 10) est intégrée au backend pour les besoins du déploiement.

Incohérence à vérifier : Le cahier des charges demande explicitement un export au format `.h5` ou `.pt`. Le projet utilise le format `.keras` (format natif de Keras 3, successeur recommandé du format `.h5` par la bibliothèque elle-même), ni `.h5` ni `.pt` (ce dernier étant spécifique à PyTorch, non utilisé dans ce projet qui repose entièrement sur TensorFlow/Keras). Aucun fichier `.h5` ou `.pt` n'a été retrouvé dans `ai/saved_models/`. Ce choix technique n'a pas été reconsidéré dans le cadre de la rédaction de ce rapport, conformément à la consigne de ne modifier ni le code ni le modèle ; il est signalé ici comme écart au cahier des charges plutôt que dissimulé.

3.16 Limites

- **broken_small** : rappel de seulement 0,5, cette classe de défaut étant régulièrement confondue avec **broken_large** (voir 9 et 11).
- **Domain shift** : le modèle a été entraîné et évalué exclusivement sur des images issues du dataset MVTec AD ; sa capacité de généralisation à des images provenant d'un domaine différent (autre éclairage, autre fond, autre appareil photo) n'a pas été établie dans ce projet.
- **Généralisation** : le jeu de test repose sur un nombre restreint d'images sources réellement uniques par classe, ce qui limite la portée statistique des conclusions tirées du 3.2.

CHAPITRE 4

Architecture de la solution

4.1 Architecture globale

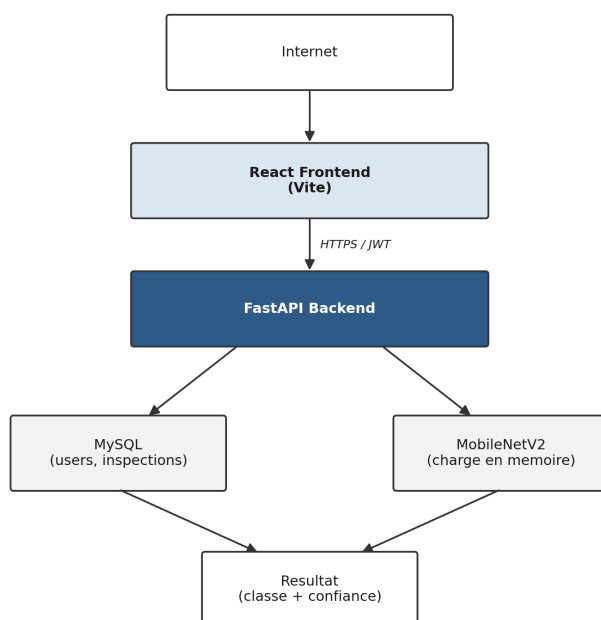


Figure 1 - Architecture globale de la plateforme VisionInspectIA

FIGURE 4.1 – Architecture globale de la plateforme VisionInspectIA.

Comme présenté à la 4.1, le frontend React communique en HTTPS/JWT avec le backend FastAPI, qui est le seul point d'entrée vers MySQL (données) et vers MobileNetV2 (IA, chargé en mémoire dans le même processus backend, et non dans un service séparé).

4.2 Frontend

Application React (bâtie avec Vite), organisée en couches : **api/** (appels HTTP centralisés), **context//hooks/** (état global : authentification, thème, notifications), **components/** (par

fonctionnalité), **pages/** (écrans), **routes/** (protection des pages privées).

4.3 Backend

API FastAPI organisée en couches strictes : **api/** (routes HTTP uniquement), **services/** (toute la logique métier), **models/** (tables SQLAlchemy), **schemas/** (validation Pydantic), **ml/** (chargement du modèle et prétraitement), **db/**, **core/**.

4.4 IA

Le modèle MobileNetV2 est chargé une seule fois en mémoire au démarrage du serveur (voir 8) et interrogé à chaque requête de prédiction, sans rechargement.

4.5 Base de données

MySQL, accédée via SQLAlchemy (ORM), détaillée au 5.

4.6 Authentification

Authentification par jeton JWT (voir 6), avec hachage des mots de passe par bcrypt.

4.7 Communication frontend/backend

Communication exclusivement via une API REST en JSON (et **multipart/form-data** pour l'upload d'image), sécurisée par HTTPS en production et protégée par une politique CORS restreinte à l'origine du frontend déployé (voir 10).

4.8 Gestion des fichiers

Les images uploadées sont stockées sur le système de fichiers du serveur backend (**backend/uploads/**), servies statiquement, et référencées en base par leur chemin relatif. Cette approche présente une limite en environnement cloud, discutée au 10 et au 11.

4.9 Sécurité

Isolation stricte des données entre utilisateurs (chaque requête est filtrée sur l'identifiant de l'utilisateur authentifié), gestion explicite des codes d'erreur HTTP (401, 400, 404), variables sensibles (**SECRET_KEY**, identifiants de base de données) exclusivement gérées via variables d'environnement, jamais versionnées.

CHAPITRE 5

Base de données

5.1 Choix de MySQL

MySQL a été retenu conformément au cahier des charges, qui l'imposait explicitement comme système de gestion de base de données pour ce projet [2].

5.2 Modèle de données

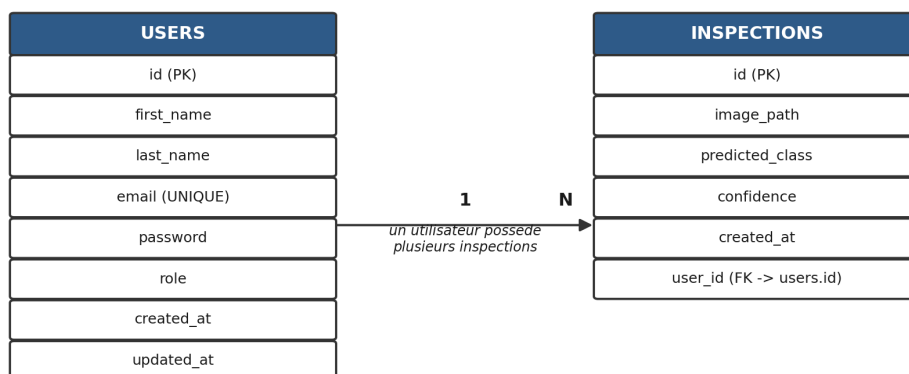


Figure 4 - Schema de la base de données MySQL (relation users 1-N inspections).
La suppression en cascade lors de DELETE /users/me est gérée par la logique applicative (user_service.py), pas par une contrainte SQL ON DELETE CASCADE.

FIGURE 5.1 – Schéma de la base de données MySQL (relation **users** 1-N **inspections**).

TABLE 5.1 – Table `users` (source : `backend/app/models/user.py`)

Champ	Type	Contrainte
<code>id</code>	Integer	Clé primaire
<code>first_name</code>	String(100)	Non nul
<code>last_name</code>	String(100)	Non nul
<code>email</code>	String(255)	Unique, indexé, non nul
<code>password</code>	String(255)	Non nul (hash bcrypt)
<code>role</code>	String(50)	Non nul, défaut "user"
<code>created_at</code>	DateTime	Automatique
<code>updated_at</code>	DateTime	Automatique

TABLE 5.2 – Table `inspections` (source : `backend/app/models/inspection.py`)

Champ	Type	Contrainte
<code>id</code>	Integer	Clé primaire
<code>image_path</code>	String(500)	Non nul
<code>predicted_class</code>	String(100)	Non nul
<code>confidence</code>	Float	Non nul
<code>created_at</code>	DateTime	Automatique
<code>user_id</code>	Integer	Clé étrangère → <code>users.id</code> , non nul

Relation : `users` (1) — (N) `inspections`.

5.3 Contraintes

Clé étrangère `inspections.user_id` → `users.id`; contrainte d'unicité sur `users.email`.

5.4 Isolation des utilisateurs

Toute requête portant sur les inspections filtre systématiquement sur l'identifiant de l'utilisateur authentifié (extrait du JWT), empêchant un utilisateur d'accéder aux données d'un autre (vérifié en production, voir 9).

5.5 Gestion des inspections

La suppression d'un compte utilisateur entraîne, au niveau de la logique applicative (`user_service.py`), la suppression des inspections associées ainsi que des fichiers image correspondants sur le disque, avant la suppression de la ligne `users`.

Point important : Il ne s'agit **pas** d'une cascade déclarée au niveau du schéma SQL (`ON DELETE CASCADE`), mais d'une suppression orchestrée par le code applicatif.

5.6 Déploiement de MySQL sur Railway

En production, la base de données est un service MySQL managé par Railway (plugin officiel), accessible uniquement depuis le réseau privé du projet (non exposé publiquement). Les tables sont créées automatiquement au démarrage du backend via l'appel de la fonction `init_db()` (voir 10, correction apportée pendant le déploiement).

Alembic est configuré dans le projet (`backend/alembic/`) mais **aucune migration versionnée n'a été générée** à ce jour — le répertoire `backend/alembic/versions/` ne contient qu'un fichier `.gitkeep`. Le schéma est actuellement géré exclusivement via `Base.metadata.create_all()`, sans historique de migrations.

CHAPITRE 6

Backend FastAPI

6.1 Présentation de FastAPI

FastAPI a été retenu, parmi les deux options proposées par le cahier des charges (FastAPI ou Flask), pour sa validation de données native via Pydantic, sa génération automatique de documentation interactive (Swagger/ReDoc), et son support natif de l'asynchrone [1].

6.2 Architecture backend

```
1 app /
2 |-- core/          # Configuration, securite (hash, JWT)
3 |-- api/           # Routes HTTP uniquement
4 |-- db/            # Connexion et session SQLAlchemy
5 |-- models/        # Tables SQLAlchemy
6 |-- schemas/       # Validation Pydantic
7 |-- services/      # Toute la logique metier
8 |-- ml/            # Chargement du modele et pretraitement
9 '-- utils/        # Fonctions utilitaires (fichiers, PDF)
```

6.3 Configuration

La configuration (`app/core/config.py`) est centralisée via `pydantic-settings`, permettant une surcharge complète par variables d'environnement (base de données, JWT, CORS, port d'écoute), sans valeur sensible codée en dur.

6.4 Authentification JWT

Un jeton JWT (algorithme HS256) est émis à la connexion et doit être fourni via l'en-tête `Authorization: Bearer <token>` pour toute route protégée. Sa durée de validité est configurable (`ACCESS_TOKEN_EXPIRE_MINUTES`).

6.5 Gestion des utilisateurs

Inscription, connexion, consultation et modification du profil, changement de mot de passe, suppression de compte.

6.6 Upload

L'upload d'image se fait en `multipart/form-data`, avec validation du format et de la taille du fichier avant tout traitement (`backend/app/utils/file_utils.py`).

6.7 Pipeline de prédiction

Détaillé au 8.

6.8 Historique

Liste, détail et suppression des inspections, filtrées sur l'utilisateur courant.

6.9 Dashboard

Statistiques agrégées calculées par requête SQL (`COUNT`, `SUM` conditionnel, `AVG`) sur MySQL, jamais pré-calculées ni stockées (`backend/app/services/dashboard_service.py`).

6.10 PDF

Génération de rapport PDF en mémoire (ReportLab), sans écriture de fichier temporaire sur le disque (`backend/app/utils/pdf_utils.py`).

6.11 Sécurité

Voir 4 et 5.

6.12 API REST

TABLE 6.1 – Endpoints de l'API REST (source : `backend/app/api/v1/router.py`)

Méthode	Endpoint	Auth	Fonction
POST	<code>/api/v1/auth/register</code>	Non	Création de compte
POST	<code>/api/v1/auth/login</code>	Non	Connexion, émission du JWT
GET	<code>/api/v1/auth/me</code>	Oui	Profil de l'utilisateur courant
POST	<code>/api/v1/auth/logout</code>	Oui	Déconnexion
PUT	<code>/api/v1/users/me</code>	Oui	Modification du profil
PUT	<code>/api/v1/users/me/password</code>	Oui	Changement de mot de passe
DELETE	<code>/api/v1/users/me</code>	Oui	Suppression du compte
POST	<code>/api/v1/predictions/predict</code>	Oui	Upload d'image + prédiction
GET	<code>/api/v1/history</code>	Oui	Liste des inspections
GET	<code>/api/v1/history/{id}</code>	Oui	Détail d'une inspection
DELETE	<code>/api/v1/history/{id}</code>	Oui	Suppression d'une inspection
GET	<code>/api/v1/dashboard/statistics</code>	Oui	Statistiques agrégées
GET	<code>/api/v1/reports/{inspection_id}</code>	Oui	Génération du rapport PDF

Chaque groupe de routes dispose également d'un endpoint `GET /health` sans logique métier associée.

CHAPITRE 7

Frontend React

7.1 Choix de React

React a été retenu parmi les deux options proposées par le cahier des charges (React ou Angular) [4].

7.2 Architecture frontend

```
1 src/  
2 |-- api/           # Appels HTTP centralises  
3 |-- components/    # Composants reutilisables, par fonctionnalite  
4 |-- pages/         # Ecrans  
5 |-- context/       # Etat global (Auth, Theme, Toast)  
6 |-- hooks/         # Hooks personnalisés  
7 |-- routes/        # Routage et protection des pages privées  
8 |-- layouts/       # Mise en page commune  
9 '-- utils/         # Statistiques dashboard, export CSV
```

7.3 Routing

Géré par React Router (`src/routes/AppRoutes.jsx`), avec protection des pages privées (redirection vers `/login` en l'absence de session valide).

7.4 AuthContext

Contexte React centralisant l'état d'authentification (token, utilisateur courant), consommé par l'ensemble de l'application via un hook dédié (`useAuth`).

7.5 Axios

Client HTTP centralisé (`src/api/axiosClient.js`), injectant automatiquement le JWT dans l'en-tête `Authorization` de chaque requête, et gérant globalement l'expiration de session (redirection automatique en cas de réponse 401).

7.6 Login / Register

Pages dédiées, consommant respectivement `POST /auth/login` et `POST /auth/register`.

7.7 Dashboard

Page affichant les statistiques agrégées et les graphiques.

7.8 Prediction

Page d'upload et d'affichage du résultat de prédiction.

7.9 History

Page listant les inspections, avec recherche, filtres, tri et pagination.

7.10 Profile

Page de gestion du profil (édition, changement de mot de passe, suppression de compte).

7.11 PDF et CSV

Téléchargement du rapport PDF depuis l'historique ; export CSV de l'historique, généré entièrement côté client.

7.12 Notifications

Centre de notifications (`ToastContext`).

7.13 Dark mode

Basculement de thème clair/sombre, persisté dans le `localStorage` du navigateur (`ThemeContext`).

7.14 Responsive design

Interface conçue pour s'adapter aux différentes tailles d'écran (desktop, tablette, mobile).

7.15 UX/UI — limite importante

Important : L'environnement de développement utilisé pour ce projet ne disposait pas de navigateur graphique. Le rendu visuel effectif (mise en page, comportement du mode

sombre, comportement responsive réel) **n’a pas pu être vérifié visuellement** au cours du développement ni de la rédaction de ce rapport. Les fonctionnalités correspondantes ont été validées uniquement par revue du code source et par des tests fonctionnels au niveau de l’API (voir 9). Aucune affirmation de type « redesign moderne validé visuellement » n’est donc formulée dans ce rapport au-delà de ce qui a été effectivement vérifié.

7.16 Fonctionnalités de la plateforme

TABLE 7.1 – Fonctionnalités implementées et leur statut de vérification

Fonctionnalité	Fonctionnement	Résultat
Register	POST /auth/register, mot de passe haché (bcrypt)	Compte créé, vérifié fonctionnel en local et en production
Login	POST /auth/login, émission d’un JWT	JWT valide retourné, vérifié fonctionnel
Logout	POST /auth/logout	Vérifié fonctionnel
Dashboard	GET /dashboard/statistics, graphiques	Statistiques cohérentes avec les données réelles de MySQL
Prediction	Upload → POST /predictions/predict	Fonctionnel sur les 4 classes; limite connue sur <code>broken_small</code>
History	GET /history, recherche, filtres, tri, pagination	Fonctionnel
Inspection details	GET /history/{id}, modal de détail	Fonctionnel, isolation entre utilisateurs vérifiée
PDF	GET /reports/{inspection_id}, généré en mémoire	Fonctionnel, y compris en dégradé sans image
CSV	Génération côté client (Blob)	Fonctionnel
Notifications	Centre de notifications (ToastContext)	Implémenté; vérification visuelle non réalisée
Dark mode	Bascule de thème, persistée en <code>localStorage</code>	Implémenté; vérification visuelle non réalisée
Profile	PUT /users/me	Testé en production, fonctionnel
Change password	PUT /users/me/password	Testé en production, fonctionnel
Delete account	DELETE /users/me, suppression en cascade	Testé en production, cascade vérifiée

CHAPITRE 8

Pipeline de prédiction IA

Le pipeline de prédiction, identique en développement et en production, se déroule comme suit (source : `backend/app/services/prediction_service.py`) :

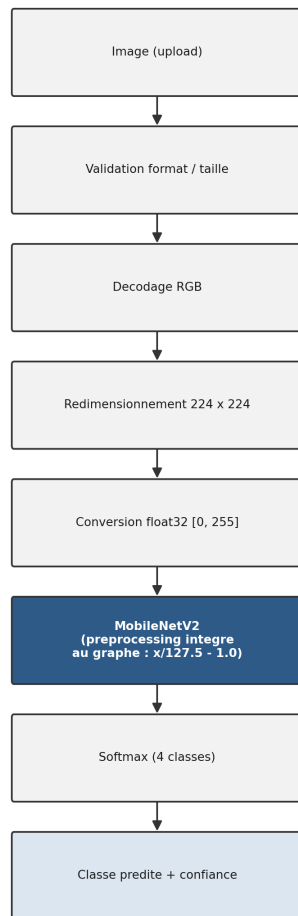


Figure 2 - Pipeline interne du modèle MobileNetV2 (`best_model.keras`).
La normalisation MobileNetV2 est intégrée au modèle, non réappliquée par le backend.

FIGURE 8.1 – Pipeline interne du modèle MobileNetV2 (`best_model.keras`).

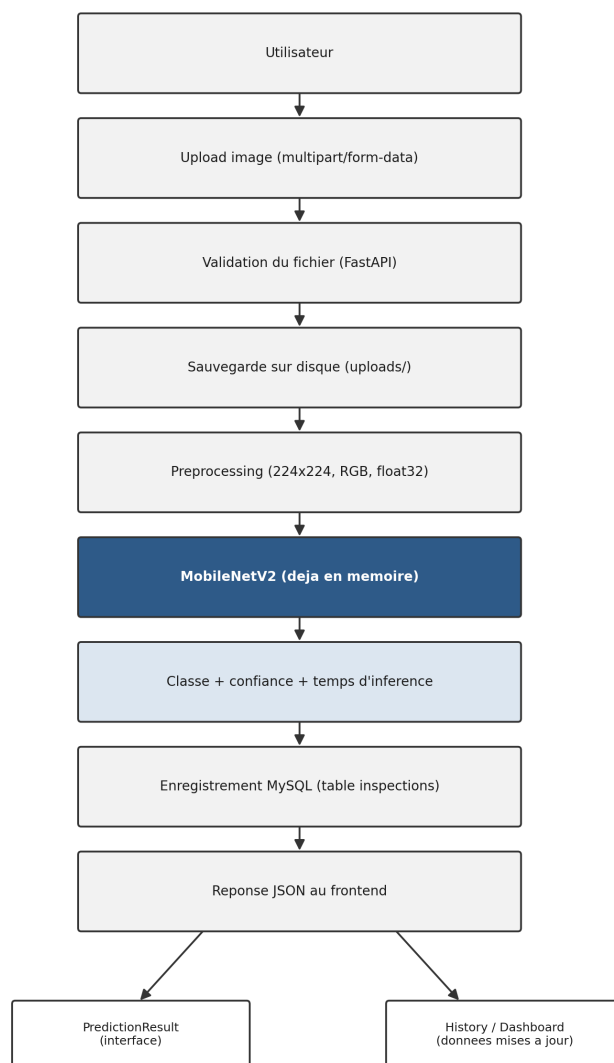


Figure 3 - Pipeline de prediction : flux complet depuis l'upload d'une image jusqu'à l'enregistrement du resultat et sa restitution au frontend.

FIGURE 8.2 – Pipeline de prédiction : flux applicatif complet depuis l'upload jusqu'à la réponse.

Comme le montre la 8.2, les étapes sont :

1. upload de l'image (multipart/form-data) ;
2. validation du format et de la taille du fichier ;
3. sauvegarde sur disque (`backend/uploads/`) ;
4. prétraitement (`224×224`, RGB, `float32`) ;
5. inférence MobileNetV2 (modèle déjà en mémoire) ;
6. calcul de la classe prédite et du score de confiance (softmax) ;
7. enregistrement de l'inspection en base MySQL ;
8. réponse JSON au frontend (classe, confiance, temps d'inférence).

Chargement du modèle. Le fichier `best_model.keras` est chargé une seule fois en mémoire au démarrage du serveur (fonction `lifespan` de `backend/app/main.py`), via un singleton

(**ModelLoader**). Il n'est jamais rechargé par la suite, chaque requête de prédiction réutilisant l'instance déjà en mémoire.

Temps d'inférence. Le temps d'inférence pure (appel `model.predict()` uniquement, modèle déjà chargé) est mesuré à chaque prédiction via `time.perf_counter()` et retourné au frontend dans la réponse JSON (champ `inference_time_ms`), sans être persisté en base de données. La valeur mesurée lors du benchmark (3) est d'environ 9 ms ; les temps mesurés en production, incluant le trajet HTTP complet, sont présentés au 10.

Point de conception important : Le preprocessing MobileNetV2 ($x/127,5 - 1,0$) est intégré directement au graphe du modèle sauvegardé (couche `MobileNetPreprocess`). Le backend ne réapplique donc **pas** une seconde normalisation lors du prétraitement — ce point a été vérifié explicitement pour éviter de fausser les prédictions (voir 2).

CHAPITRE 9

Tests et validation

9.1 Tests backend

Démarrage du serveur, disponibilité de la documentation interactive (`/docs`, `/redoc`), connexion à MySQL.

9.2 Tests frontend

Build de production (`npm run build`) : 2505 modules transformés, 0 erreur, vérifié à plusieurs reprises au cours du projet.

9.3 Tests API

Ensemble des endpoints listés au 6.1, testés par des requêtes HTTP réelles (et non simulées).

9.4 Tests IA

Prédiction testée sur une image de chacune des quatre classes, en local puis en production.

9.5 Tests sécurité

Accès sans JWT, JWT invalide, tentative d'accès aux données d'un autre utilisateur — tous testés et confirmés bloqués (codes 401/404 appropriés), y compris en production.

9.6 Tests base de données

Vérification du schéma, des contraintes (clé étrangère, unicité de l'email), et de la cohérence entre les fichiers du dossier `uploads/` et les lignes correspondantes en base.

9.7 Tests PDF

Génération vérifiée sur des inspections réelles, y compris en dégradé lorsque le fichier image n'est plus disponible.

9.8 Tests CSV

Génération vérifiée sur des données réelles d'historique.

9.9 Tests end-to-end

Scénario complet exécuté en local puis en production : inscription → connexion → upload → prédiction → historique → tableau de bord → PDF → modification du profil → déconnexion.

9.10 Tests en production

TABLE 9.1 – Tests en production (requêtes HTTP réelles contre les URLs publiques de déploiement)

Fonctionnalité	Test	Résultat
Frontend accessible	GET / et routes SPA	✓ 200 sur toutes les routes
Backend accessible	GET /	✓ 200
Swagger / ReDoc	GET /docs, GET /redoc	✓ 200 / 200
Register	POST /auth/register	✓ 201
Login	bon et mauvais mot de passe	✓ 200 / 401
Prediction — good	Upload réel	✓ correct, confiance 99,4 %
Prediction — broken_large	Upload réel	✓ correct, confiance 99,8 %
Prediction — broken_small	Upload réel	! incorrect , prédit broken_large (80,5 %)
Prediction — contamination	Upload réel	✓ correct, confiance 99,3 %
History	GET /history	✓ cohérent après redéploiement
Dashboard	GET /dashboard/statistics	✓ cohérent avec les inspections réelles
PDF	GET /reports/{id}	✓ 200, même si image absente
Profile	PUT /users/me	✓ modifié et confirmé
Change password	PUT /users/me/password	✓ ancien rejeté, nouveau accepté
Delete account	DELETE /users/me	✓ token révoqué, email réutilisable
Logout	POST /auth/logout	✓ 200
Isolation entre utilisateurs	accès à une ressource d'un autre utilisateur	✓ 404
Sans JWT / JWT invalide	GET /history	✓ 401 / 401
Fichier invalide	upload .zip	✓ 400, message explicite
CORS	requête réelle depuis le domaine du frontend	✓ origine exacte, pas de wildcard

Sur broken_small : Ce résultat n'est pas caché : il est conforme à la limite déjà identifiée lors du benchmark (3) et discutée au 11.

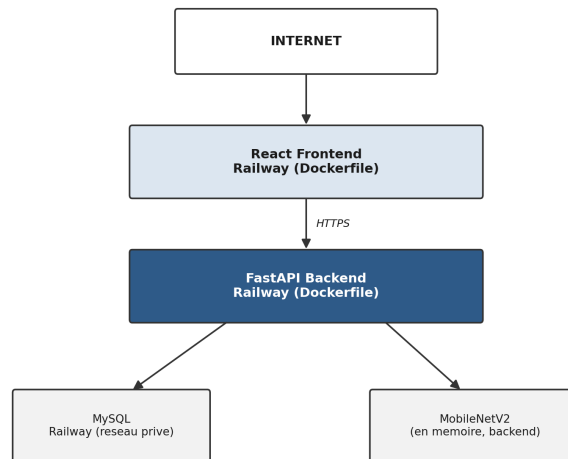
CHAPITRE 10

Déploiement

10.1 Pourquoi Railway

Parmi les trois plateformes proposées par le cahier des charges (Render, Railway, Hugging Face Spaces), Railway [3] a été retenue après comparaison : c'est la seule à proposer une base de données MySQL managée nativement (Render ne propose pas de MySQL managé, seulement PostgreSQL/Redis ; Hugging Face Spaces n'est pas conçu pour une application full-stack avec authentification et base de données relationnelle). Railway permet ainsi d'héberger backend, frontend et base de données au sein d'un unique projet.

10.2 Architecture de production



Authentification : JWT | Communication : REST API | MySQL : reseau prive uniquement

Figure 12 - Architecture de déploiement en production sur Railway.
Le modele MobileNetV2 est integre au service backend, ce n'est pas un service separe.

FIGURE 10.1 – Architecture de déploiement en production sur Railway.

10.3 Docker

Chaque service (backend, frontend) est déployé via un `Dockerfile` explicite (`backend/Dockerfile`, `frontend/Dockerfile`). Ce choix a été fait après l'échec du build automatique par défaut de Railway (Railpack) sur la structure en monorepo du projet (voir 10.12).

10.4 Backend

Service Railway distinct, `Root Directory` = `backend/`, construit à partir d'un `Dockerfile` (image `python:3.13-slim`, installation des dépendances via `pip`, lancement par `uvicorn`).

10.5 Frontend

Service Railway distinct, `Root Directory` = `frontend/`, construction en deux étapes (build Vite avec `VITE_API_URL` injecté comme argument de build Docker, puis service statique via le paquet `serve`).

10.6 MySQL

Base de données provisionnée via le plugin officiel MySQL de Railway, accessible uniquement en réseau privé au sein du projet.

10.7 Variables d'environnement

Les variables suivantes sont définies au niveau de chaque service Railway, jamais versionnées dans le dépôt Git : `DATABASE_HOST`, `DATABASE_PORT`, `DATABASE_USER`, `DATABASE_PASSWORD`, `DATABASE_NAME` (backend), `SECRET_KEY`, `ALGORITHM`, `ACCESS_TOKEN_EXPIRE_MINUTES`, `DEBUG`, `CORS_ORIGINS` (backend), `VITE_API_URL` (frontend, en tant qu'argument de build).

Sécurité : Aucune valeur réelle de ces variables n'est reproduite dans ce rapport ni dans le dépôt (`backend/.env.example` ne contient que des placeholders).

10.8 MobileNetV2 en production

Le backend embarque sa propre copie du modèle (`backend/app/ml/model_files/best_model.keras`), rendant son déploiement indépendant du dossier `ai/`. L'identité de cette copie avec le modèle original a été vérifiée par comparaison d'empreinte SHA-256 : aucune modification des poids n'a été introduite. Les journaux du service en production confirment le chargement effectif du modèle au démarrage (message `MobileNetV2 loaded successfully`).

10.9 URLs

- **Frontend** : `frontend-production-bcff.up.railway.app`
- **Backend** : `visioninspectia-production.up.railway.app`
- **Swagger** : `visioninspectia-production.up.railway.app/docs`
- **ReDoc** : `visioninspectia-production.up.railway.app/redoc`

10.10 Tests en production

Détaillés au 9 (9.1).

10.11 Performances

Mesures réellement observées en production (et non des estimations) :

- **Cold start** (premier appel après démarrage, incluant le *warm-up* TensorFlow) : environ **1,9 à 2,3 secondes**.
- **Inférence à chaud** (appels suivants, temps HTTP complet incluant upload, validation, prétraitement, inférence et écriture en base) : environ **170 à 210 ms**.

Ces deux valeurs sont à distinguer du temps d'inférence pure mesuré lors du benchmark (≈ 9 ms, 3), qui ne couvre que l'appel au modèle sur une image déjà prétraitée, modèle déjà chargé.

10.12 Problèmes rencontrés

Le déploiement a nécessité plusieurs itérations correctives, documentées ici pour leur valeur pédagogique :

1. **Échec de détection par Railpack** : le builder par défaut de Railway ne parvenait pas à détecter l'application depuis la racine du dépôt (structure en monorepo).
2. **Dépendance du backend au dossier ai/** : le chargement du modèle référençait initialement `ai/saved_models/...` comme dossier frère de `backend/`, incompatible avec un déploiement restreint à `backend/` seul.
3. **Échec de détection de la commande de démarrage** par Railpack, y compris avec la commande définie côté service.
4. **Erreur d'ordonnancement des couches de build** avec un fichier de configuration Railpack personnalisé.
5. **Absence d'expansion de la variable \$PORT** lors du démarrage du conteneur.
6. **Configuration figée lors d'un redéploiement** : la commande `redploy` de Railway rejouait une configuration capturée au moment du build précédent.
7. **Absence d'appel à `init_db()`** au démarrage, entraînant une erreur « table inexistante » sur une base neuve.

8. **Stockage non persistant** des images uploadées entre redéploiements (système de fichiers éphémère du conteneur).

10.13 Solutions appliquées

- **Points 1–2** : le backend a été rendu autonome par la duplication vérifiée (empreinte SHA-256 identique) du fichier modèle et de la couche de prétraitement personnalisée directement dans `backend/app/ml/`.
- **Points 3–5** : abandon du builder automatique au profit d'un **Dockerfile explicite** par service.
- **Point 6** : déclenchement d'un nouveau commit Git (plutôt qu'un simple **redeploy**) pour forcer la prise en compte de la configuration à jour.
- **Point 7** : ajout de l'appel à `init_db()` dans la fonction `lifespan()` de `backend/app/main.py`
 - modification idempotente, sans impact sur l'environnement de développement local existant.
- **Point 8** : documentée explicitement comme limitation connue plutôt que traitée par l'ajout d'une solution de stockage persistant, jugée hors du périmètre raisonnable pour une démonstration de stage.

CHAPITRE 11

Résultats, discussion, limitations et perspectives

11.1 Résultats IA

Voir 3.2 et 3.3 (3).

11.2 Benchmark

Le benchmark met en évidence l'apport déterminant du transfer learning par rapport à un entraînement from scratch dans ce contexte de données limitées.

11.3 Analyse par classe

Les classes **good** et **broken_large** sont détectées avec un rappel parfait par MobileNetV2 sur le jeu de test. Les classes **broken_small** et **contamination** présentent un rappel plus faible (0,5 et 0,525 respectivement), traduisant une difficulté de discrimination plus importante pour ces défauts.

Le cas **broken_small**

Cette classe constitue la principale limite de performance identifiée dans ce projet, confirmée à la fois sur le jeu de test et lors des tests en production (9, où l'unique test réalisé sur cette classe a été mal classé). La cause la plus probable, au regard des données disponibles, est le faible nombre d'images sources réelles pour cette classe (22 images brutes au total, réparties entre validation et test), rendant la distinction avec **broken_large** difficile à apprendre de façon robuste.

Domain shift

Aucune évaluation formelle de la robustesse du modèle à un changement de domaine (images hors MVTec AD) n'a été réalisée dans ce projet. Ce point reste une perspective (voir 11.7).

11.4 Résultats applicatifs et production

L'ensemble des fonctionnalités listées au cahier des charges a été implémenté et testé fonctionnellement (7, 9). L'application déployée a été testée de bout en bout avec des requêtes réelles contre les URLs publiques, confirmant son fonctionnement effectif au-delà de l'environnement de développement local.

11.5 Discussion

Il convient de distinguer clairement deux niveaux de performance évalués dans ce projet :

- **La performance sur le jeu de test du dataset MVTec AD**, qui reste, pour les classes `good`, `broken_large` et `contamination`, à un niveau satisfaisant compte tenu du volume de données disponible, mais qui repose sur un jeu de test partiellement constitué d'images dupliquées à partir d'un faible nombre de sources uniques, ce qui limite la portée statistique de ces chiffres.
- **La capacité de généralisation au monde réel**, c'est-à-dire à des photographies de bouteilles prises dans des conditions différentes de celles du dataset MVTec AD (éclairage, arrière-plan, appareil photo, chaîne de production réelle), qui **n'a pas été évaluée** dans ce projet. Les tests réalisés, y compris en production, utilisent exclusivement des images issues du même dataset que celui utilisé pour l'entraînement. Aucune affirmation de performance en conditions industrielles réelles ne peut donc être formulée à partir des résultats obtenus.

11.6 Limitations

TABLE 11.1 – Limitations identifiées

Limitation	Description
Dataset limité	Le dataset MVTec AD (<i>bottle</i>) ne compte que 292 images réelles, avec seulement 20 à 22 images par classe de défaut.
Difficulté sur <code>broken_small</code>	Rappel de 0,5 sur le jeu de test, confirmé par un test en production; confusion récurrente avec <code>broken_large</code> .
Domain shift	La capacité de généralisation à des images hors du dataset MVTec AD n'a pas été évaluée.
Manque de photos terrain	Aucune photographie prise dans des conditions industrielles réelles n'a été utilisée pour l'évaluation.
Stockage uploads/ non persistant sur Railway	Le système de fichiers du conteneur backend est éphémère; une image uploadée est perdue au redéploiement suivant (confirmé par test). Les données en base ne sont pas affectées.
Cold start TensorFlow	Premier appel après démarrage sensiblement plus lent (≈ 2 s) que les appels suivants.
Limites du tier Railway	Le projet est hébergé sur l'offre gratuite/d'essai de Railway, sans garantie de disponibilité prolongée.
Absence de vérification visuelle	Le rendu effectif de l'interface (mode sombre, responsive) n'a pas pu être vérifié dans un navigateur réel.
Alembic non exploité	Alembic est configuré mais aucune migration versionnée n'a été générée.

11.7 Perspectives

Les pistes suivantes sont présentées comme des travaux futurs envisageables, **non implémentés** dans la version actuelle du projet :

- enrichissement du dataset avec des photographies réelles supplémentaires, notamment pour la classe `broken_small` ;
- collecte d'images en conditions industrielles réelles, pour évaluer et améliorer la robustesse au domain shift ;
- techniques d'augmentation plus avancées (ex. augmentation ciblée sur les classes les plus faibles) ;
- mise en place d'un mécanisme de détection *out-of-distribution* (rejet des images ne correspondant à aucune classe connue) ;
- migration vers un stockage cloud persistant (type S3) pour les images uploadées ;
- optimisation du modèle pour un déploiement plus léger (par exemple conversion TensorFlow Lite) ;
- mise en place de migrations Alembic versionnées ;
- amélioration du pipeline de déploiement (automatisation CI/CD, environnement de staging).

CHAPITRE 12

Conclusion générale

Ce stage a permis de mener un projet complet, de la préparation d'un dataset d'images industrielles jusqu'au déploiement d'une application web fonctionnelle sur une infrastructure cloud réelle, en couvrant les quatre phases définies par le cahier des charges TechSkillHub.

Sur le plan de l'intelligence artificielle, quatre architectures de classification d'images ont été entraînées et comparées dans des conditions expérimentales rigoureusement identiques. Cette comparaison a mis en évidence l'apport déterminant du transfer learning face à un entraînement from scratch dans un contexte de données limitées, et a conduit à la sélection de MobileNetV2, retenu non pas comme le modèle le plus performant dans l'absolu, mais comme le meilleur compromis entre performance, taille et rapidité d'inférence pour une intégration dans une application web.

Sur le plan applicatif, une plateforme complète a été développée : une API REST FastAPI structurée en couches, une base de données MySQL, et une interface React couvrant l'ensemble des fonctionnalités attendues (authentification, prédiction, historique, tableau de bord, export PDF/CSV, gestion du profil).

Sur le plan du déploiement, l'application a été réellement mise en ligne sur Railway et testée de bout en bout par des requêtes effectives contre les URLs publiques, démarche qui a permis d'identifier et de corriger plusieurs problèmes concrets propres à un environnement de production (dépendances de build, gestion du port, initialisation de la base de données), documentés dans ce rapport à titre d'apprentissage technique.

Ce travail présente des limites clairement identifiées : une performance de détection inégale selon les classes de défaut (notamment sur `broken_small`), une évaluation de généralisation au monde réel non réalisée, et une contrainte de stockage propre à l'environnement de déploiement choisi. Ces limites ne remettent pas en cause la validité de la démarche mise en œuvre, mais délimitent précisément le périmètre dans lequel les résultats obtenus peuvent être interprétés.

Ce stage a permis de mettre en pratique, sur un cas d'usage industriel concret, l'ensemble d'une chaîne de traitement en intelligence artificielle appliquée : de la donnée brute à l'application déployée, en passant par l'expérimentation contrôlée et l'évaluation critique des résultats.

Bibliographie

- [1] FastAPI — Documentation officielle. <https://fastapi.tiangolo.com>.
- [2] MySQL — Documentation officielle. <https://dev.mysql.com/doc/>.
- [3] Railway — Documentation officielle. <https://docs.railway.app>.
- [4] React — Documentation officielle. <https://react.dev>.
- [5] Paul Bergmann, Michael Fauser, David Sattlegger, and Carsten Steger. MVTEC AD — A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection. Dataset used in this project (bottle category). [RÉFÉRENCE À COMPLÉTER : année/venue exacts de publication non vérifiés dans le cadre de ce rapport].
- [6] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep Residual Learning for Image Recognition. Architecture comparée dans le benchmark (Chapitre 3). [RÉFÉRENCE À COMPLÉTER : année/venue exacts de publication non vérifiés dans le cadre de ce rapport].
- [7] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. MobileNetV2 : Inverted Residuals and Linear Bottlenecks. Architecture retenue pour ce projet. [RÉFÉRENCE À COMPLÉTER : année/venue exacts de publication non vérifiés dans le cadre de ce rapport].
- [8] Mingxing Tan and Quoc Le. EfficientNet : Rethinking Model Scaling for Convolutional Neural Networks. Architecture comparée dans le benchmark (Chapitre 3). [RÉFÉRENCE À COMPLÉTER : année/venue exacts de publication non vérifiés dans le cadre de ce rapport].
- [9] TechSkillHub. AI/ML Internship Task Assignment. Document de cadrage du stage, référence TSH/A5E51B1E.

ANNEXE A

Architecture globale

Voir 4 et 4.1.

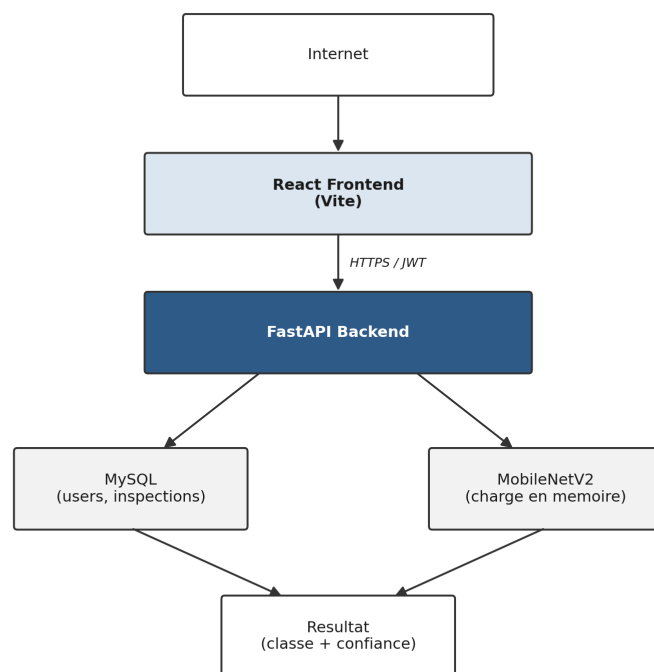


Figure 1 - Architecture globale de la plateforme VisionInspectIA

FIGURE A.1 – Rappel : architecture globale de la plateforme VisionInspectIA.

ANNEXE B

Endpoints de l'API

Voir 6.1 (6) pour la liste complète des endpoints réellement exposés par l'API REST (source : `backend/app/api/v1/router.py`).

ANNEXE C

Base de données

Voir 5, 5.1, 5.2 et 5.1.

ANNEXE D

Résultats IA

Voir 3.2 et 3.3 (3).

Correspondance avec le cahier des charges

TABLE D.1 – Correspondance exigences du cahier des charges / réalisation

Exigence	Réalisation	Preuve
Dataset	MVTec AD, catégorie bottle	2
4 modèles	CNN / MobileNetV2 / EfficientNetB0 / ResNet50	3
Accuracy	Oui	3.2
Precision	Oui	3.2
Recall	Oui	3.2
F1-score	Oui	3.2
Matrice de confusion	Oui	3.2
Courbe ROC	Non disponible — aucun fichier trouvé dans <code>ai/results/</code>	3
React	Oui	7
FastAPI	Oui	6
MySQL	Oui	5
Authentification	Oui (JWT)	6
Upload	Oui	8
Historique	Oui	7
PDF	Oui	6, 7
Dashboard	Oui	7
Déploiement cloud	Railway	10
Format modèle <code>.h5/.pt</code>	Non — format <code>.keras</code> utilisé à la place	3
Présentation (PPT)	[À COMPLÉTER] — non produit dans le cadre de ce rapport	—
	LaTeX	
Vidéo de démonstration	[À COMPLÉTER] — non enregistrée à ce jour	—

Important : La courbe ROC n'est pas présentée comme existante : aucun fichier correspondant n'a été retrouvé dans `ai/results/`, et aucune courbe n'a été inventée pour combler cette absence.

ANNEXE E

Captures de la plateforme

Aucune capture d'écran n'a été produite dans le cadre de la rédaction de ce rapport (environnement de développement sans navigateur graphique). Les captures suivantes restent à réaliser avant la finalisation du rapport :

RE À AJOUTER Login

RE À AJOUTER Register

RE À AJOUTER Dashboard

RE À AJOUTER Prediction (interface d'upload)

RE À AJOUTER Résultat de prédiction (classe, confiance, temps d'inférence)

RE À AJOUTER History

RE À AJOUTER Inspection details (modal)

RE À AJOUTER PDF généré

RE À AJOUTER Export CSV

RE À AJOUTER Profile

RE À AJOUTER Dark mode

1. Swagger — réalisable immédiatement à l'URL <https://visioninspectia-production.up.railway.app/docs>

RE À AJOUTER Railway — service frontend

RE À AJOUTER Railway — service backend

ANNEXE F

Déploiement

Voir 10, URLs listées en 10.9, problèmes et solutions en 10.12.